

Nome: Guilherme Borges de Pádua Barbosa
Nº USP: 15653045

Trabalho 1 - AED

Todos os itens de 1 a 5 foram feitos e os códigos estão rodando corretamente.

Aplicação do Código:

- O programa vai ser baseado em um menu de registros de pessoas, cada usuário será registrado de acordo com o seu nome, idade e número de matrícula. A estrutura 'Pessoa' foi identificada nos dois arquivos .h como:

```
struct Pessoa{  
    string nome;  
    int idade;  
    long int matricula;  
};
```

- A ordenação da árvore AVL é por ordem alfabética, ou seja, comparação dos nomes de entrada. E da mesma forma, a função Hash utiliza o nome de cada usuário para conseguir a chave:

```
else  
    if(x.nome < pA->entry.nome){  
        searchInsert(x, pA->leftNode, h);
```

```
HashKey HashTable::transform(string nome){  
    HashKey chave = 0;  
    for(int i = 0; i < size(nome); i++){  
        chave = (chave*128 + nome[i])%TAMANHO; //função hash  
    }  
    return chave;  
}
```

- O menu terá as mesmas características, apenas se diferenciando pelo modo de armazenamento dos dados:

Menu - AVL:

```
1 - Inserir  
2 - Remover  
3 - Buscar  
4 - Imprimir  
5 - Sair
```

Menu - Tabela Hash:

```
1 - Inserir  
2 - Remover  
3 - Buscar  
4 - Imprimir  
5 - Sair
```

Resultado dos Testes:

- Inserção:
 - Tabela Hash:

```
guilherme@debian:~/Code/C++/AED/Trabalho1$ g++ *.cpp -o main
guilherme@debian:~/Code/C++/AED/Trabalho1$ ./main
```

```
Menu - Tabela Hash:
```

```
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
1
```

```
Insira seu nome:
```

```
Guilherme Borges
```

```
Insira sua idade:
```

```
19
```

```
Insira sua matrícula:
```

```
15653045
```

```
Menu - Tabela Hash:
```

```
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
4
```

```
[Nome: Guilherme Borges - Idade: 19 - Matrícula: 15653045].
```

- Árvore Balanceada:

```
Insira seu nome: Guilherme Borges
Insira sua idade: 19
Insira sua matrícula: 15653045

Menu - AVL:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
4
Guilherme Borges
```

(A impressão da árvore balanceada apenas imprime o nome por causa da formatação)

- Remoção

- Tabela Hash:

```
[Nome: Guilherme Borges - Idade: 19 - Matrícula: 15653045].
```

```
Menu - Tabela Hash:
```

```
1 - Inserir  
2 - Remover  
3 - Buscar  
4 - Imprimir  
5 - Sair  
2
```

```
Insira a pessoa para remover.
```

```
Nome: Guilherme Borges
```

```
Idade: 19
```

```
Matrícula: 15653045
```

```
p: 1
```

```
Pessoa removida com sucesso.
```

```
Menu - Tabela Hash:
```

```
1 - Inserir  
2 - Remover  
3 - Buscar  
4 - Imprimir  
5 - Sair  
4
```

```
Tabela está vazia.
```

(Na ordem: Impressão da tabela com o usuário presente, remoção do usuário e impressão da tabela vazia.)

- Árvore Balanceada:

```
Guilherme Borges

Menu - AVL:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
2

Insira a pessoa para remover.
Nome: Guilherme Borges
Idade: 19
Matrícula: 15653045
Usuário removido com sucesso!

Menu - AVL:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
4
```

(Na ordem: Impressão da árvore com o usuário presente, remoção do usuário e impressão da árvore vazia.)

- Consulta

- Tabela Hash:

```
1
[Nome: Guilherme Borges - Idade: 19 - Matrícula: 15653045].

Menu - Tabela Hash:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
3

Insira a pessoa para buscar
Nome: Guilherme Borges
Idade: 19
Matrícula: 15653045

Esse nome está na tabela, e sua posição na lista é: 1.
```

- Árvore Balanceada:

```
1
Guilherme Borges

Menu - AVL:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
3

Insira a pessoa para busacr.
Nome: Guilherme Borges
Idade: 19
Matrícula: 15653045
Esse nome está na árvore.
```

Erros:

```
Menu - AVL:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
2

Insira a pessoa para remover.
Nome: Julio Rodolfo
Idade: 22
Matrícula: 1435462
Não foi possível remover o usuário.
```

(Remoção de usuário inexistente - AVL)

```
Menu - AVL:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
3

Insira a pessoa para busacr.
Nome: Julio Rodolfo
Idade: 22
Matrícula: 1435462
Esse nome não está na árvore.
```

(Busca de usuário inexistente - AVL)

```
Menu - Tabela Hash:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
3

Insira a pessoa para buscar
Nome: Julio Rodolfo
Idade: 22
Matrícula: 1435462

Esse nome não está na tabela.
```

(Busca de usuário inexistente - Tabela Hash)

```
Menu - Tabela Hash:
1 - Inserir
2 - Remover
3 - Buscar
4 - Imprimir
5 - Sair
2

Insira a pessoa para remover.
Nome: Julio Rodolfo
Idade: 22
Matrícula: 1435462
Pessoa não encontrada.
```

(Remoção de usuário inexistente - Tabela Hash)

Vantagens e Desvantagens:

Árvores AVL são complexas de serem construídas porém conseguem manter faixas de tempo menores para inserção e remoção $O(\log n)$, e também mantém os dados ordenados, elas podem ser vantajosas em casos que a ordenação é algo realmente necessário para a resolução do problema e em casos de um número maior de registros.

Já as tabelas Hash são uma solução mais simples do problema e a busca direta de registros é imediata por causa da chave, porém podem ser mais lentas em inserção e remoção. Por construir um array estático, ocupa mais espaço de memória em comparação com a árvore balanceada.

Códigos:

AVL.cpp

```
/**
 *   Nome:Guilherme Borges de Pádua Barbosa
 *   N° USP: 15653045
 */

#include <iostream>
#include <iomanip>
#include "AVL.h"
#include <string>

using namespace std;

AVLTree::AVLTree() {
    root = NULL;
}

AVLTree::~~AVLTree() {
    clear();
}

void AVLTree::clear() {
    clear(root);
}

void AVLTree::clear(TreePointer &t) {
```

```

    if (t != NULL) {
        clear(t->leftNode);
        clear(t->rightNode);
        delete t;
        t = nullptr;
    }
}

void AVLTree::searchInsert(TreeEntry x) {
    bool h = false;
    searchInsert(x, root, h);
}

void AVLTree::searchInsert(TreeEntry x, TreePointer &pA, bool &h) {
    TreePointer pB, pC;

    if (pA == NULL) {
        pA = new TreeNode;
        h = true;
        pA->entry = x;
        pA->count = 1;
        pA->leftNode = pA->rightNode = NULL;
        pA->bal = 0;
    }

    else
        if (x.nome < pA->entry.nome) {
            searchInsert(x, pA->leftNode, h);
            if (h) {
                switch (pA->bal) {
                    case -1: pA->bal = 0; h = false; break;
                    case 0: pA->bal = 1; break;
                    case 1:
                        pB = pA->leftNode;
                        if (pB->bal == 1) { //LL
                            pA->leftNode = pB->rightNode;
                            pB->rightNode = pA;
                            pA->bal = 0;
                            pA = pB;
                        }
                    }
            }
        }
}

```

```

        else{ //LR
            pC = pB->rightNode;
            pB->rightNode = pC->leftNode;
            pC->leftNode = pB;
            pA->leftNode = pC->rightNode;
            pC->rightNode = pA;
            if(pC->bal == 1) pA->bal = -1; else pA->bal
= 0;

            if(pC->bal == -1) pA->bal = 1; else pB->bal
= 0;

            pA = pC;
        }
        pA->bal = 0; h = false;
    }
}
else
    if(x.nome > pA->entry.nome){
        searchInsert(x, pA->rightNode, h);
        if(h){ // subárvore direita cresceu
            switch (pA->bal){
                case +1: pA->bal = 0; h = false; break;
                case 0: pA->bal = -1; break;

                case -1: pB = pA->rightNode;
                    if(pB->bal == -1){ // rotação RR
                        pA->rightNode = pB->leftNode; pB->leftNode =
pA;

                        pA->bal = 0; pA = pB;
                    }
                    else{ // rotação RL
                        pC = pB->leftNode; pB->leftNode =
pC->rightNode;

                        pC->rightNode = pB; pA->rightNode =
pC->leftNode;

                        pC->leftNode = pA;
                        if(pC->bal == -1) pA->bal = +1; else pA->bal =
0;

                        if(pC->bal == +1) pB->bal = -1; else pB->bal =
0;

```

```

        pA = pC;
    }
    pA->bal = 0; h = false;
} // switch
}
}
else pA->count++;
}

bool AVLTree::remove(TreeEntry x) {
    bool h=false;
    return remove(x,root,h);
}

//-----
bool AVLTree::remove(TreeEntry x, TreePointer &p, bool &h){
    TreePointer q;
    bool removeu;

    if(p == NULL) // x não encontrado
        return false;

    if(x.nome < p->entry.nome){
        removeu = remove(x,p->leftNode,h);
        if(h)
            balanceL(p,h);
        return removeu;
    }
    else
        if(x.nome > p->entry.nome){
            removeu = remove(x,p->rightNode,h);
            if(h)
                balanceR(p,h);
            return removeu;
        }
    else{ // remover p->
        q = p;
        if(q->rightNode == NULL){
            p = q->leftNode;
            h = true;

```

```

    }
    else
        if(q->leftNode == NULL){
            p = q->rightNode;
            h = true;
        }
        else{
            removeMin(q, q->rightNode, h);
            if(h)
                balanceR(p, h);
        }

        delete q;
        delete p;
        p = NULL;
        return true; // x removido
    }
}

void AVLTree::removeMin(TreePointer &q, TreePointer &r, bool &h){
    if(r->leftNode != NULL){
        removeMin(q, r->leftNode, h);
        if(h)
            balanceL(r, h);
    }
    else{
        q->entry = r->entry;
        q->count = r->count;
        q = r;
        r = r->rightNode;
        h = true;
    }
}

void AVLTree::balanceL(TreePointer &pA, bool &h)
{
    TreePointer pB, pC;
    int balB, balC;

    // subárvore esquerda encolheu

```

```

switch (pA->bal)
{
    case +1:
        pA->bal = 0;
        break;

    case 0:
        pA->bal = -1;
        h = false;
        break;

    case -1:
        pB = pA->rightNode;
        balB = pB->bal;

        if (balB <= 0) // rotação RR
        {
            pA->rightNode = pB->leftNode;
            pB->leftNode = pA;

            if (balB == 0)
            {
                pA->bal = -1;
                pB->bal = +1;
                h = false;
            }
            else
            {
                pA->bal = 0;
                pB->bal = 0;
            }

            pA = pB;
        }
        else // rotação RL
        {
            pC = pB->leftNode;
            balC = pC->bal;

            pB->leftNode = pC->rightNode;

```

```

        pC->rightNode = pB;

        pA->rightNode = pC->leftNode;
        pC->leftNode = pA;

        if (balC == -1)
            pA->bal = +1;
        else
            pA->bal = 0;

        if (balC == +1)
            pB->bal = -1;
        else
            pB->bal = 0;

        pA = pC;
        pC->bal = 0;
    }
    break;
}

void AVLTree::balanceR(TreePointer &pA, bool &h){
    TreePointer pB, pC;
    int balB, balC;

    // subárvore direita encolheu
    switch (pA->bal)
    {
        case -1:
            pA->bal = 0;
            break;

        case 0:
            pA->bal = +1;
            h = false;
            break;

        case +1:

```

```
pB = pA->leftNode;
balB = pB->bal;

if (balB >= 0) // rotação LL
{
    pA->leftNode = pB->rightNode;
    pB->rightNode = pA;

    if (balB == 0)
    {
        pA->bal = +1;
        pB->bal = -1;
        h = false;
    }
    else
    {
        pA->bal = 0;
        pB->bal = 0;
    }

    pA = pB;
}
else // rotação LR
{
    pC = pB->rightNode;
    balC = pC->bal;

    pB->rightNode = pC->leftNode;
    pC->leftNode = pB;

    pA->leftNode = pC->rightNode;
    pC->rightNode = pA;

    if (balC == +1)
        pA->bal = -1;
    else
        pA->bal = 0;

    if (balC == -1)
        pB->bal = +1;
```

```

        else
            pB->bal = 0;

            pA = pC;
            pC->bal = 0;
        }
        break;
    }
}

bool AVLTree::search(TreeEntry x){
    return rSearch(x, root);
}

//-----
--
bool AVLTree::rSearch(TreeEntry x, TreePointer &t){
    if (t == NULL)
        return false; // x não encontrado

    if (x.nome < t->entry.nome)
        return rSearch(x, t->leftNode);
    else if (x.nome > t->entry.nome)
        return rSearch(x, t->rightNode);
    else
        return true; // x.nome == t->entry.nome
}

void AVLTree::print(){
    print(root,0);
}

void AVLTree::print(TreePointer &t, int s){
    int i;
    if(t != NULL){
        print(t->rightNode, s+3); // escreve subárvore direita
        for(i=1; i<=s; i++)
            cout << " "; // espaços
        cout << setw(6) << t->entry.nome << endl; // escreve a raiz
        print(t->leftNode, s+3); // escreve subárvore esquerda
    }
}

```

```
}  
}
```

AVL.h

```
1  /**
2  *   Nome:Guilherme Borges de Pádua Barbosa
3  *   N° USP: 15653045
4  */
5  #include <string>
6  #include "pessoa.h"
7  using namespace std;
8
9  #ifndef AVL_H
10 #define AVL_H
11
12 typedef Pessoa TreeEntry;
13
14 class AVLTree {
15 public:
16     AVLTree();
17     ~AVLTree();
18     void searchInsert(TreeEntry x);
19     bool remove(TreeEntry x);
20     bool search(TreeEntry x);
21     void clear();
22     void print();
23
24 private:
25     struct TreeNode;
26     typedef TreeNode * TreePointer;
27
28     struct TreeNode{
29         TreeEntry entry;
30         int count;
31         int bal;
32         TreePointer leftNode, rightNode;
33     };
34
35     TreePointer root;
36
37     void clear(TreePointer &t);
38     void searchInsert(TreeEntry x, TreePointer &pA, bool &h);
39     bool remove(TreeEntry x, TreePointer &p, bool &h);
40     void removeMin(TreePointer &q, TreePointer &r, bool &h);
41     void balanceR(TreePointer &pA, bool &h);
42     void balanceL(TreePointer &pA, bool &h);
43     void print(TreePointer &t, int s);
44     bool rSearch(TreeEntry x, TreePointer &t);
45 };
46
47
48 #endif
49
```

Hash.cpp

```
/**
 *   Nome:Guilherme Borges de Pádua Barbosa
 *   N° USP: 15653045
 */

#include <iostream>
#include <string>
#include "Hash.h"
#include "lista.h"

HashTable::HashTable() {
    tabela = new List[TAMANHO]; //cria um array de listas com o tamanho
definido
}

HashTable::~HashTable() {
    clear();
}

void HashTable::clear() {
    for(int i = 0; i < TAMANHO; i++){
        tabela[i].clear(); //usa o método de lista 'clear' para a tabela
inteira
    }
}

HashKey HashTable::transform(string nome) {
    HashKey chave = 0;

    for(int i = 0; i < size(nome); i++){
        chave = (chave*128 + nome[i])%TAMANHO; //função hash
    }
    return chave;
}

void HashTable::insert(HashEntry pessoa) {
    HashKey chave = transform(pessoa.nome);
```

```

        tabela[chave].insert(1, pessoa); //Insere no começo da lista de posição
        chave
    }

int HashTable::search(HashEntry &pessoa) {
    HashKey chave = transform(pessoa.nome); //Descobre a chave Hash
    int resultado = tabela[chave].search(pessoa.matricula); //Encontra
    usando o método da lista, e modifica o x

    return(resultado == 0 ? 0 : resultado); //Retorna 0 se não for
    encontrado ou a posição na lista se for
}

void HashTable::remove(HashEntry &pessoa) {
    HashKey chave = transform(pessoa.nome);
    int posicao = tabela[chave].search(pessoa.matricula); //encontra a
    posicao do item com base na matrícula, na lista de indice 'chave'

    if(posicao != 0) {
        tabela[chave].remove(posicao, pessoa); //remove o item, se
        encontrado
        cout << "Pessoa removida com sucesso." << endl;
    }
    else
        cout << "Pessoa não encontrada." << endl;
}

void HashTable::print() {
    if(empty()) {
        cout << "Tabela está vazia." << endl;
        return;
    }

    for(int i = 0; i < TAMANHO; i++) {
        if(!tabela[i].empty()) {
            tabela[i].print();
        }
    }
}

```

```
bool HashTable::empty() {  
    bool resultado = true;  
  
    for(int i = 0; i < TAMANHO; i++){  
        if(!tabela[i].empty())  
            resultado = false; //Se alguma das listas não estiver vazia, a  
tabela também não estará  
    }  
  
    return resultado;  
}
```

Hash.h

```
/**
 *   Nome:Guilherme Borges de Pádua Barbosa
 *   N° USP: 15653045
 */

#include <string>
#include "lista.h"
using namespace std;

#ifndef HASH_H
#define HASH_H

typedef Pessoa HashEntry;
typedef int HashKey;

class HashTable {
public:
    HashTable();
    ~HashTable();
    HashKey transform(string x);
    void insert(HashEntry x);
    int search(HashEntry &x);
    void remove(HashEntry &x);
    void clear();
    void print();
    bool empty();

private:
    const int TAMANHO = 701; //Tamanho da tabela
    List *tabela; //cria um array de listas
};

#endif
```

Lista.cpp

```
/**
 *   Nome:Guilherme Borges de Pádua Barbosa
 *   N° USP: 15653045
 */

#include <iostream>
#include "lista.h"
using namespace std;

List::List() {
    head = NULL;
    count = 0;
}

List::~~List() {
    clear();
}

bool List::empty() {
    return (head == NULL);
}

bool List::full() {
    return false;
}

void List::setPosition(int p, ListPointer &current) {
    int i;
    current = head;
    for(i = 2; i <= p; i++)
        current = current->nextNode;
}

void List::insert(int p, ListEntry x) {
    ListPointer newNode, current;

    if (p < 1 || p > count+1) {
        cout << "Posição Inválida.";
    }
}
```

```

        abort();
    }

    newNode = new ListNode;
    newNode->entry = x;

    if(p == 1){
        newNode->nextNode = head;
        head = newNode;
    }
    else{
        setPosition(p-1, current);
        newNode->nextNode = current->nextNode;
        current->nextNode = newNode;
    }

    count++;
}

void List::remove(int p, ListEntry &x){
    ListPointer node, current;
    if (p < 1 || p > count){
        cout << "Posição inválida";
        abort();
    }

    if(p == 1){
        node = head;
        head = node->nextNode;
    }
    else{
        setPosition(p-1, current);
        node = current->nextNode;
        current->nextNode = node->nextNode;
    }

    x = node->entry;
    delete node;
    count--;
}

```

```

void List::retrieve(int p, ListEntry &x){
    ListPointer q;

    if(p < 1 || p > count)
        abort();

    setPosition(p,q);
    x = q->entry;
}

void List::clear(){
    ListPointer q;
    while (head != NULL){
        q = head;
        head = head->nextNode;
        delete q;
    }
}

int List::size(){
    return count;
}

int List::search(long int x){
    int p = 1;
    ListPointer q = head;

    while(q != NULL && q->entry.matricula != x){
        q = q->nextNode;
        p++;
    }

    return (q == NULL ? 0 : p);
}

void List::print(){
    ListPointer q = head;
    cout << "[Nome: " << q->entry.nome << " - Idade: " << q->entry.idade <<
" - Matrícula: " << q->entry.matricula;

```

```
q = q->nextNode;

while(q != NULL){
    cout << ", Nome: " << q->entry.nome << " - Idade: " <<
q->entry.idade << " - Matrícula: " << q->entry.matricula;
    q = q->nextNode;
}

cout << "]" << endl;
}
```

Lista.h

```
/**
 *   Nome:Guilherme Borges de Pádua Barbosa
 *   N° USP: 15653045
 */

#include <string>
#include "pessoa.h"
using namespace std;

#ifndef LISTA_H
#define LISTA_H

typedef Pessoa ListEntry;

class List{
public:
    List();
    ~List();
    void insert(int p, ListEntry x);
    void remove(int p, ListEntry &x);
    void retrieve(int p, ListEntry &x);
    bool empty();
    bool full();
    void clear();
    int size();
    int search(long int x);
    void print();

private:
    struct ListNode;
    typedef ListNode * ListPointer;
    struct ListNode{
        ListEntry entry;
        ListPointer nextNode;
    };

    ListPointer head;
    int count;
```

```
void setPosition(int p, ListPointer &current);  
};  
  
#endif
```

main.cpp

```
/**
 *   Nome:Guilherme Borges de Pádua Barbosa
 *   N° USP: 15653045
 */

#include <iostream>
#include <iomanip>
#include "AVL.h"
#include "Hash.h"
#include "lista.h"
#include <string>
using namespace std;

void menuHash(HashTable &h){
    cout << "\nMenu - Tabela Hash:" << "\n1 - Inserir" << "\n2 - Remover"
    << "\n3 - Buscar" << "\n4 - Imprimir" << "\n5 - Sair" << endl;
    int op, resultado;
    Pessoa p;
    cin >> op;

    switch(op){
        case 1:
            cout << "\nInsira seu nome: " << endl;
            cin.ignore();
            getline(cin, p.nome);

            cout << "\nInsira sua idade: " << endl;
            cin >> p.idade;

            cout << "\nInsira sua matrícula: " << endl;
            cin >> p.matricula;

            h.insert(p);
            menuHash(h);
            break;

        case 2:
            cout << "\nInsira a pessoa para remover." << endl;
```

```
    cout << "Nome: ";
    cin.ignore();
    getline(cin, p.nome);

    cout << "Idade: ";
    cin >> p.idade;

    cout << "Matrícula: ";
    cin >> p.matricula;

    h.remove(p);

    cout << endl;
    menuHash(h);
    break;

case 3:
    cout << "\nInsira a pessoa para buscar" << endl;

    cout << "Nome: ";
    cin.ignore();
    getline(cin, p.nome);

    cout << "Idade: ";
    cin >> p.idade;

    cout << "Matrícula: ";
    cin >> p.matricula;

    resultado = h.search(p);

    cout << endl;

    if(resultado != 0)
        cout << "Esse nome está na tabela, e sua posição na lista
é: " << resultado << ".\n\n";
    else
        cout << "Esse nome não está na tabela.\n\n";
    menuHash(h);
```

```

        break;

    case 4:
        h.print();
        menuHash(h);
        break;

    case 5:
        break;

    default:
        cout << "Entrada Inválida" << endl;
        abort();
    }
}

void menuArvore(AVLTree &t){
    cout << "\nMenu - AVL:" << "\n1 - Inserir" << "\n2 - Remover" << "\n3 -
Buscar" << "\n4 - Imprimir" << "\n5 - Sair" << endl;

    int op;
    string nome;
    Pessoa p;
    cin >> op;

    switch(op){
        case 1:
            cout << "\nInsira seu nome: ";
            cin.ignore();
            getline(cin, p.nome);

            cout << "Insira sua idade: ";
            cin >> p.idade;

            cout << "Insira sua matrícula: ";
            cin >> p.matricula;

            t.searchInsert(p);
            menuArvore(t);
            break;

```

```
case 2:
    cout << "\nInsira a pessoa para remover." << endl;

    cout << "Nome: ";
    cin.ignore();
    getline(cin, p.nome);

    cout << "Idade: ";
    cin >> p.idade;

    cout << "Matrícula: ";
    cin >> p.matricula;

    if(t.remove(p))
        cout << "Usuário removido com sucesso!" << endl;
    else
        cout << "Não foi possível remover o usuário." << endl;

    menuArvore(t);
    break;

case 3:
    cout << "\nInsira a pessoa para busacr." << endl;

    cout << "Nome: ";
    cin.ignore();
    getline(cin, p.nome);

    cout << "Idade: ";
    cin >> p.idade;

    cout << "Matrícula: ";
    cin >> p.matricula;

    if(t.search(p))
        cout << "Esse nome está na árvore.\n" << endl;
    else
        cout << "Esse nome não está na árvore.\n" << endl;
    menuArvore(t);
    break;
```

```

        case 4:
            t.print();
            menuArvore(t);
            break;

        case 5:
            break;

        default:
            cout << "Entrada Inválida" << endl;
            abort();
    }
}

int main() {
    AVLTree Tree;
    //menuArvore(Tree);

    HashTable tabela;
    menuHash(tabela);
    return 0;
}

```

pessoa.h

```

1  #include <string>
2  #ifndef PESSOA_H
3  #define PESSOA_H
4
5  struct Pessoa{
6      std::string nome;
7      int idade;
8      long int matricula;
9  };
10
11 #endif

```